Design Airbnb Booking Platform

我們要設計 Airbnb 網站，**聚焦於訪客（guest）的訂房流程**。

功能性需求（Functional Requirements）

1. 使用者可以依據 **location（地點）** 與 **dates（日期）** 搜尋 listings（房源）。為簡化問題，location 將以 **city name（城市名稱）來表示**。
2. 使用者可以查看 **home details（房源詳細資訊）**。省略 images、reviews，只保留文字資訊。
3. 使用者可以在指定日期 **book a home（預訂房源）**。

非範圍（Out of Scope）

1. Host admin（房東後台）
2. Payments process（付款流程）
3. Recommendations（推薦系統）

非功能性需求（Non-Functional Requirements）

1. 系統在搜尋與查看房源時應優先考量 **availability（可用性）**，但在訂房時應優先考量 **consistency（一致性）**，不可發生 double booking（重複訂房）。
2. 系統需具備良好的 **scalability（可擴展性）**，能承受旺季（peak season）的流量高峰。
3. Search latency 必須低於 **500ms**。

API 設計（API Design）

1. 搜尋房源（Searching for listings）

buildmoat.org

```
GET /home/search?city={city}&startDate={start_date}&endDate
={end_date}&pageSize={page_size}&page={page_number}
→ Home[]
```

2. 取得房源詳細資訊（Getting home details）

```
GET /home/:homeId
```

```
{
  "id": "",
  "city": "",
  "address": "",
  "type": "",
  "amenities": []
}
```

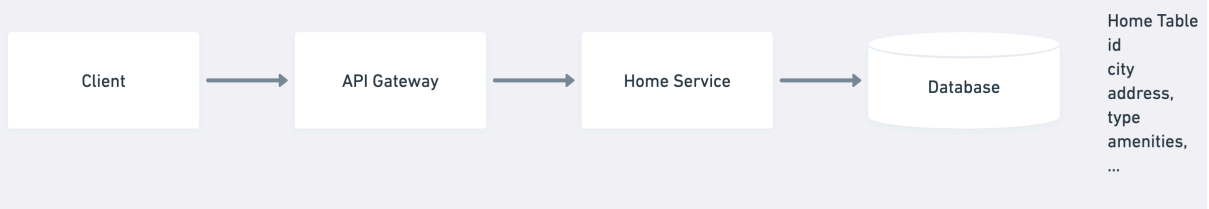
3. 預訂房源（Booking a home）

```
POST /home/book?homeId={home_id}&startDate={start_date}&end
Date={end_date}
```

buildnoat.org

High-Level Design

1. 房源詳細頁查看流程（Home Detail Viewing Flow）

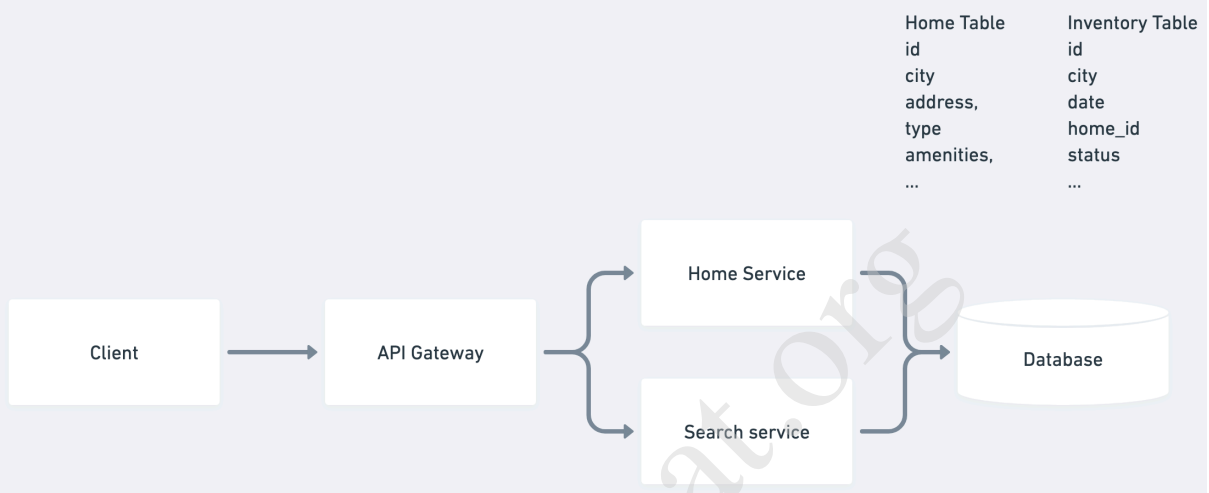


1. 使用者送出 GET request，依 `home_id` 取得房源詳細資訊。
2. API Gateway 將請求轉發至 Home Service（房源服務）。

3. Home Service 查詢 Home Table（房源資料表），並將房源詳細資訊回傳給使用者。

> 此流程為 read-heavy + availability-first，可透過 cache（例如 Redis / CDN）進一步降低 latency。

2. 房源搜尋流程（Home Search Flow）

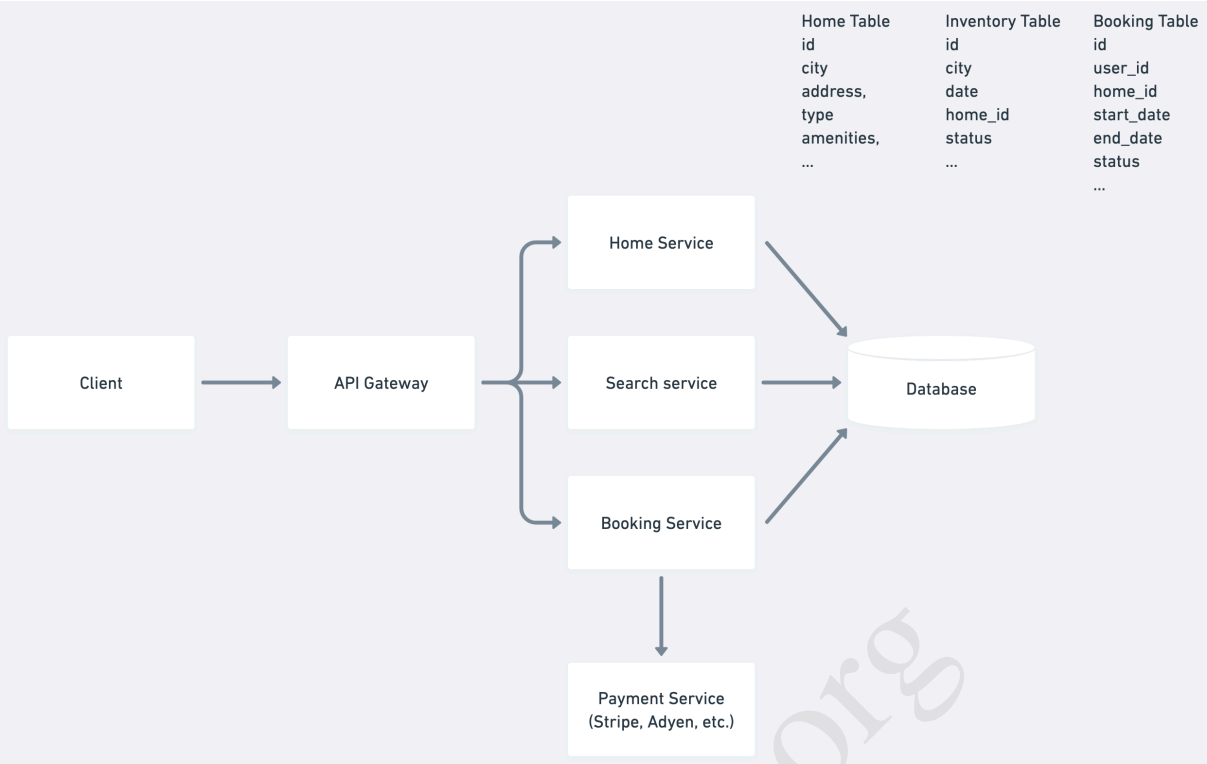


1. 使用者送出 GET request，指定 city、date range 與 pagination 參數進行搜尋。
2. API Gateway 將請求轉發至 Search Service（搜尋服務）。
3. Search Service 查詢 Inventory Table（庫存表），條件包含：
   - 指定 date range
   - 狀態為 `available`
   1. Inventory Table 以 **(home_id, date)** 為一筆資料。
   2. 因此需要定期為每個 home 預先產生未來一段時間的 dates（例如未來 6–12 個月）。
4. Search Service 對結果進行 pagination，並將 listings 回傳給使用者。

> 此設計能讓 search query 轉為 簡單的 range scan + filter，以換取低 latency（< 500ms），代價是較高的寫入與儲存成本。

3. 訂房流程（Booking Flow）

buildnoat.org



1. 使用者送出 POST request 至 Booking Service（訂房服務），嘗試在指定日期區間預訂 home。
2. Booking Service：
   - 檢查 Inventory Table，確認該 home 在 date range 內皆為 `available`。
   - 新增一筆 Booking Table（訂單表）資料，狀態為 `created`。
3. Booking Service 將使用者導向第三方 Payment Service（付款服務）頁面輸入付款資訊。
   a. 基於 compliance（法規遵循）考量，除非公司有專門的支付部門，否則通常會委託第三方支付服務（例如 Stripe、Adyen）。
   b. 這類服務具備更高等級的安全標準來保存使用者的付款資訊。
4. 當 Payment Service 回傳付款成功：
   - Booking Service 將 Booking Table 狀態更新為 `paid`。
   - 同時將 Inventory Table 對應日期的狀態更新為 `booked`。

> 這條路徑屬於 consistency-first，後續 Deep Dive 會需要說明：
>
> - 如何避免 double booking
> - inventory update 與 booking 狀態更新的 transactional / conditional write 設計

buildmoat.org

深入探討（Deep Dives）

1. 如何改善多人同時訂房的使用者體驗（concurrent booking）？

當多位使用者嘗試在 **overlapping dates（重疊日期）** 訂同一個 home 時，最終只會有一位成功。

目前流程中，使用者完成 payment 後，Booking Service 才發現 inventory 已被更新為 `booked`，導致：

- 使用者體驗極差（付款成功卻訂不到）
- 系統還需要額外處理 void payments（退款）

方案比較（Options）

1.（不佳）Pessimistic Locking（悲觀鎖）

對每次 booking 執行一個 DB transaction，對目標 inventory rows 加鎖（通常透過 `SELECT FOR UPDATE`），直到收到 Payment Service 的回應才釋放。在這段期間，所有其他嘗試對相同 rows 執行 `SELECT FOR UPDATE` 的 transaction 都必須等待 lock 被釋放。

1. 無論付款成功或失敗，lock 都會被釋放。
2. 需要一個機制來處理使用者中途離開的情況，以便釋放 lock。

為什麼這在實務上很糟？

1. 長時間持有 transaction（例如 5 分鐘的 lock 期間）會大量消耗 DB 資源，並增加 lock contention 與 deadlock 的風險。
    - 例如：User A 想預訂 09/01–09/05，User B 想預訂 09/04–09/06。User A 先取得 09/01 和 09/02 的 lock，而 User B 先取得 09/04–09/05 的 lock。兩人互相等待對方釋放 lock，形成 deadlock。
2. PostgreSQL 等 SQL DB 並未原生支援 transaction 內的 lock timeout。要實作 timeout 需要 application-level 的管理，增加額外複雜度。

2.（較好）在 Inventory Table 加入 status 與 expiration time，搭配 cron job 回復狀態

在 Inventory Table 新增 `reserved` 狀態以及 `expiration_time` 欄位。

1. 當使用者嘗試 booking 時，inventory 狀態改為 `reserved`，`expiration_time` 設為當前時間加 10 分鐘。

buildnoat.org

2. 若使用者付款成功，狀態改為 `booked`；付款失敗則回復為 `available`。
3. 若使用者耗時過久或放棄購買，當 expiration time 到達後，狀態回復為 `available`。
4. 我們可以用一個 cron job 定期掃描 inventory 中狀態為 `reserved` 且已超過 expiration time 的 rows，將它們改回 `available`。

此方案的問題：

1. 在 ticket 過期與 cron job 實際執行之間，存在固有的時間落差（inherent delay）。
2. 如果 cron job 發生故障或延遲，可能對整個 booking 流程造成嚴重影響。

3.（最佳）將可用性判斷為邏輯狀態而非實體狀態

我們可以比 cron-based 方案做得更好。關鍵洞察在於：任何一個 ticket 的實際狀態是兩個屬性的組合——它是 `available`，或者它是 `reserved` 但 reservation 已過期。Transaction 流程變為：

1. Begin transaction。
2. 檢查當前 ticket 是否為 `AVAILABLE`，或者是 `RESERVED` 但已過期（expired）。
3. 將 ticket 更新為 `RESERVED`，expiration 設為 `now + 10 minutes`。
4. Commit transaction。

我們仍然需要 cron job 來定期掃描並更新 DB row 的實體狀態，因為過期的 reservation 不會自動被翻轉回 `available`。但以這種方式設計，即使 cron job 的掃描延遲了，我們的系統行為也不會受到影響。

> 這是一種 logical availability > physical status 的設計，是實務中非常常見的 booking 系統做法。

2. 如何改善搜尋延遲（search latency）？

目前搜尋 home 的實作會導致 full table scan。查詢語句類似：

```
SELECT home_id FROM Inventory
WHERE city = 'Honolulu'
AND start_date = ...
AND end_date = ...
```

buildmoat.org

隨著資料庫中 homes 數量不斷增長，這個查詢會越來越慢且昂貴。此外，未來很可能需要擴充支援依不同欄位查詢的功能。我們該如何提升搜尋效率？

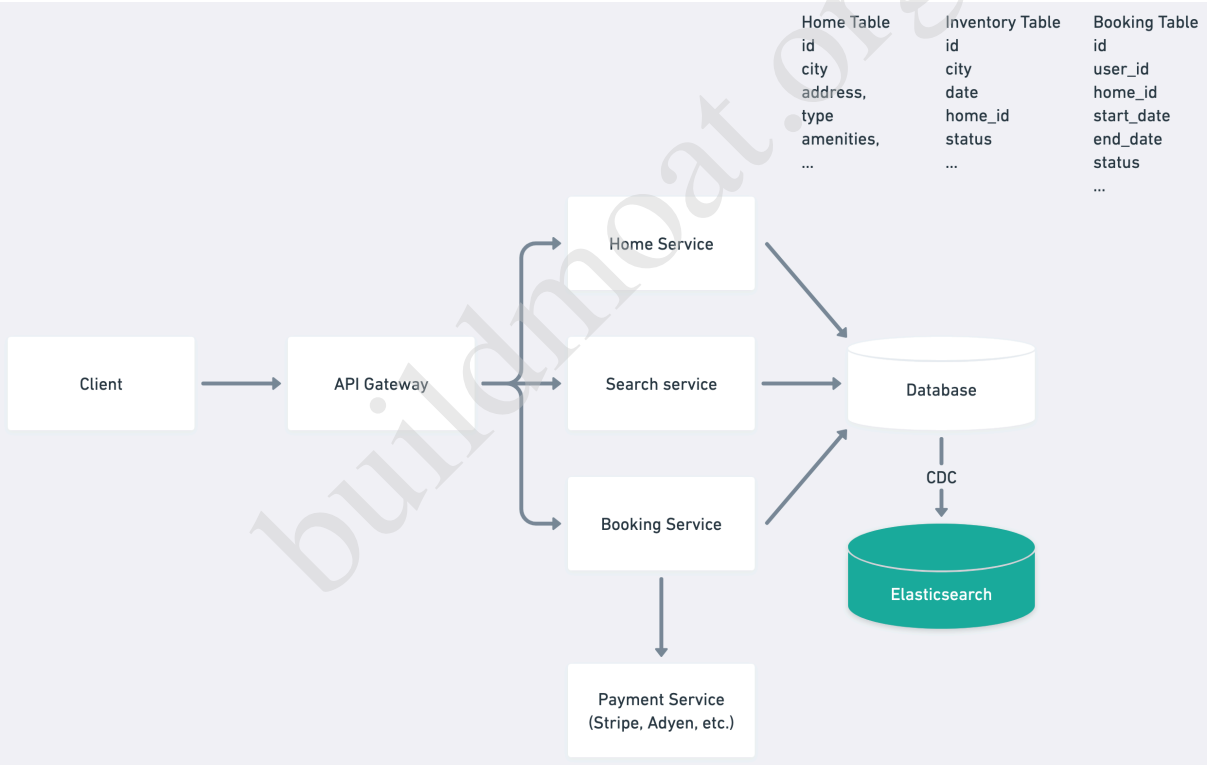
改善方案

1. 對 city、dates 等欄位建立 indexes（索引）

Indexes 透過將特定欄位的值對應到資料表中的相應 rows，來加速資料檢索。這能減少需要掃描的 row 數量，進而加速搜尋查詢。此外，我們也可以透過避免 `SELECT *` 或使用 `LIMIT` 來減少回傳的 rows 與 columns 數量，進一步優化查詢。

- 新增 indexes 會拖慢寫入操作，因為每次寫入都需要同步更新所有相關 indexes。因此需要在 index 數量與整體資料庫效能之間取得平衡。

2. 使用 Elasticsearch（全文搜尋引擎）



buildmoat.org

Elasticsearch 是一個強大的搜尋引擎，擅長全文搜尋、複雜查詢執行，以及高效處理大量流量。其核心運作機制是 **inverted indexes（倒排索引）**，這是讓它在搜尋操作上高度高效的關鍵特性。Inverted indexes 將每個唯一的詞彙對應到它所出現的 documents 或 records，使 Elasticsearch 能快速定位並檢索資料，大幅加速搜尋查詢。此外，Elasticsearch 也支援 fuzzy search，能夠對搜尋查詢與 DB 中的 terms 進行 partial match（部分匹配）。

1. 在大多數情況下，Elasticsearch 會透過 **CDC（Change Data Capture）** 連接到 authoritative data store（如 Postgres 或 DynamoDB）。
2. 保持 Elasticsearch index 與 PostgreSQL 之間的同步可能相當複雜，需要可靠的機制來確保資料一致性。
3. 維護 Elasticsearch cluster 會增加額外的基礎設施複雜度與成本。

3. 如何擴展系統以應付旺季流量（peak season traffic）？

假設我們有 10M DAU，每位使用者平均花 1 小時瀏覽網站。

```
10M × 3600 / 86400 = 400K QPS
```

- 我們的系統是 **read-heavy** 的，因此可以在 database 之上加入 cache layer，讓每個 request 不必都直接讀取資料庫。
  - 我們可以 cache home data，entry 格式為 `home_id: home_object`。由於 home data 很少變動，因此可以預期 hit rate 會很高。
  - 當使用者要存取 home data 時，server 先檢查 cache 中是否有該 key，若有則直接回傳結果。若沒有，server 從 DB 讀取資料，更新 cache 後再回傳給使用者。
  - 我們也可以設定 DB triggers 來 invalidate cache entries，這樣當 home details 被更新時，使用者不會拿到 stale data（過期資料）。
- Services 是 stateless 的，因此可以直接進行 **horizontal scaling（水平擴展）**。

4. 如何支援更通用的地理搜尋（geo search）？

我們簡化了設計，僅允許使用者依城市搜尋 homes。如果我們想擴展服務，讓使用者可以搜尋任意 landmark（地標）一定距離內的 homes，該怎麼做？

1. 整合第三方地圖 SDK / API

- Pros：方便且直接。
- Cons：成本可能較高。如果我們正在做 MVP 且使用量不大，可以先從這個方案開始。

2. Geospatial Database（例如 PostgreSQL 的 PostGIS extension）

- Pros：不需要新的 service 或外部依賴。

buildmoat.org

- Cons：實作需要較多時間。如果讀取流量很高，需要考慮加入 read replicas 或 cache 常見查詢。

3. 獨立的 cache layer 搭配 geospatial index 支援（例如 Redis）

- Pros：Cache 回應速度快，因為資料在 in-memory。
- Cons：額外的費用與維運成本來維護一個獨立的 service。對於像 Airbnb 這樣 write throughput 不高的使用場景，我會評估這個選項是否真的值得投入成本，或者其實用 Option 1 就夠了。

> 實務上，我會優先評估 Option 1 → Option 2，只有在 geo queries 成為核心瓶頸時才導入 Redis GEO。

延伸閱讀可參考：地震災害警報系統（Geospatial Index 設計）

buildmoat.org